

Cracking the AI System Design Interview

An Insider's Guide to AI System Design Interviews, with 10 Real-World Use Cases

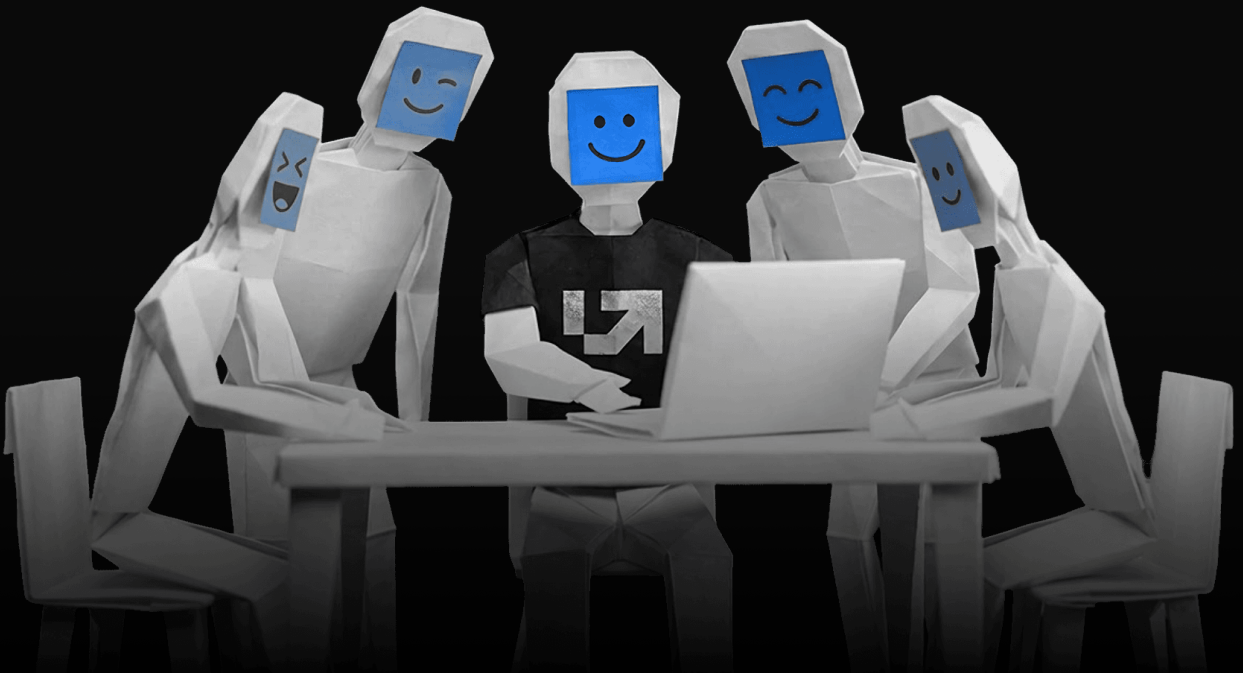
Written by

Aishwarya Naresh Reganti
Kiriti Badam

Published by

LevelUp Labs

2026 • levelup-labs.ai



Cracking the AI System Design Interview

An Insider's Guide to AI System Design Interviews, with 10 Real-World Use Cases.

Written by Aishwarya Naresh Reganti and Kiriti Badam. Published by LevelUp Labs.

Copyright © 2026 LevelUp Labs. All rights reserved.

The content, frameworks, worked case studies, and the accompanying code are the intellectual property of LevelUp Labs. Third-party materials cited throughout remain the property of their respective publishers.

This book is licensed under Creative Commons Attribution-NonCommercial 4.0 (CC BY-NC 4.0). You may share it freely with credit to LevelUp Labs. Commercial use or derivative works require prior written permission.

Before you begin. This book assumes you have basic working knowledge of AI systems and are preparing for the system design round. If you are starting from scratch, begin with our [AI Builder's Handbook](#), which builds the foundation this book stands on, then come back.

On AI and this book. We used AI as an amplifier on our own artifacts and code, to help structure and tighten the writing. The frameworks and the cases are ours, and so is the judgment behind them. Some phrasing may read as AI-assisted because it is. The thinking behind it is not.

Companion code github.com/aishwaryanr/awesome-generative-ai-guide

Contact levelup-labs.ai

Edition Public, 2026

Contents

Preface	1
Part 1: The Method	4
Chapter 1: How to Approach AI System Design Interviews	5
Part 2: The Cases	12
Chapter 2: The Customer-Support Agent	13
Chapter 3: The Enterprise Research Assistant	44
Chapter 4: Text-to-SQL Analytics	82
Chapter 5: Financial Ops Decisioning	114
Chapter 6: Document Decisioning	146
Chapter 7: The Security and Compliance Agent	180
Chapter 8: The Coding Agent	214
Chapter 9: The Voice Support Agent	249
Chapter 10: The SDR Sales Agent	283
Chapter 11: The Clinical Scribe	315
Part 3: Patterns and the Playbook	345
Chapter 12: Design Topics Across the Cases	345
Chapter 13: Running the Code	356
Chapter 14: The Interview-Day Playbook	363

Part 4: The PM and Strategy Lens	372
Chapter 15: The PM and Strategy Lens	373
Resource Index	385

Preface

This is a book about one interview round, taught through 10 real systems.

The AI system design round has become one of the rounds that decides AI hires. It is where an interviewer stops asking whether you know what RAG stands for and starts asking whether you can design a system that holds up in production: real users, real failure modes, a model that is right most of the time and wrong in ways you cannot fully predict. Most candidates have never been taught how to answer it. This book teaches it the way it is actually assessed, by walking 10 real use cases end to end on one framework you can reuse for anything.

What This Is

10 worked AI system design interviews. Each one is a real question, a full answer, and the follow-ups an interviewer pushes on when your first answer holds up. Every case is built on the same 5-layer spine, so by the 10th case you are not memorizing 10 designs, you are fluent in one way of thinking that composes into any of them.

Every case also ships **runnable, provider-agnostic code**. The companion repository is public, and each chapter points you to the exact folder so you can clone it, read it, and run it. Reading a design and running the design are different kinds of understanding, and interviews test both.

The pattern in each case chapter is the same: the question stated the way an interviewer would ask it, scoped before it is architected, the 5 layers walked for that specific problem going deepest where the weight sits, the follow-ups worked, the design anchored in a real deployment with real numbers, and a pointer to the code.

Who This Is For

AI system design is not one job's skill. There is an engineer's version of the round, and there is a product manager's and a strategist's version of the same conversation, weighted differently.

- **Engineers and forward-deployed engineers** face this as the signature technical round: architect the system out loud and defend the trade-offs.
- **Product managers** face a lighter version inside product-sense and technical-literacy rounds: does AI belong here, what is the model's role, how do you measure it, and where does a human stay in the loop.
- **Strategists** face it as the buildable path to production inside a case: scope, sequence, govern, and know where the design breaks.

The engineer's walkthrough is the spine of this book. A dedicated part near the end reframes all 10 cases through the product-and-strategy lens, so whichever seat you are interviewing for, the same 10 systems teach you what you need.

Where This Comes From

This is not generic advice. The framework and the cases are drawn from building production AI systems across dozens of enterprise deployments and

from studying how real companies at the frontier ship them, checked against direct conversations with the people who run these interview loops. The real-world anchors in each chapter are cited, including the cautionary numbers.

How the Book Is Organized

The book is 4 parts.

Part 1, The Method, is short and load-bearing. It covers what the round tests, the 5-layer spine you will use for every case, and how to scope a problem before you architect it. Read it first.

Part 2, The Cases, is the heart of the book: 10 chapters, one per system, each worked end to end.

Part 3, Patterns and the Playbook, pulls back to the cross-cutting design topics the cases teach, shows you how to run the code, and gives you an interview-day playbook for answering under pressure.

Part 4, The PM and Strategy Lens, reframes the same 10 cases for product and strategy interviews.

3 Ways to Read This

Linearly. Part 1, then the cases in order. The cases are sequenced so each foregrounds a different load-bearing layer, and the spine compounds as you go.

By case. Interviewing at a company that builds one of these systems? Read that chapter, run its code, and work the follow-ups out loud.

By role. Engineers live in Parts 1 through 3. PMs and strategists read Part 1, skim the engineer cases for the shape, and go deep in Part 4.

One thing worth saying upfront. Reading a worked case does not substitute for answering one out loud. The people who pass this round can narrate the design and its tradeoffs without notes, because they have run the reps. This book is the scaffolding and the reps. The fluency is yours to build.

Let us begin with the round itself.

LevelUp Labs

Chapter 1: How to Approach AI System Design Interviews

This is the reference chapter. It covers what the round tests, the one framework you will use to answer every case in this book, and the scoping work that comes before any architecture. The 10 cases that follow all lean on this chapter, so it is worth reading closely and returning to.

1.1 What the Round Tests

The AI system design round is where a lot of AI hires are won and lost, and it is the round candidates are least prepared for. Most preparation stops at the vocabulary, and vocabulary is not what this round tests. Knowing what a vector database is does not tell you when to reach for one, what to put around it so it fails safely, or how you would know it is working.

The interviewer is not grading a diagram. They are grading judgment under ambiguity, surfaced through a design conversation. 4 things are assessed at once.

- **Can you scope before you architect.** Strong candidates spend the first minutes narrowing the problem and surfacing assumptions. Weak candidates start drawing boxes. In AI systems the largest failures are designed in before a single prompt is written, and the round rewards the people who know that.

- **Can you reason about tradeoffs out loud.** Retrieval versus fine-tuning, more autonomy versus more control, the cost, latency, and quality triangle, evaluation as a release gate. The interviewer wants to hear you weigh options and commit, not recite a menu.
- **Do you know how it breaks.** The best answers turn to failure without being asked. How does this system produce a wrong answer, what is the blast radius, and where does a human stay in the loop. A design with no story for its own failure is an incomplete one.
- **Can you make it real.** The best answers connect to how systems ship: how you would evaluate it, what you would monitor, what you would roll back. Candidates who have built real systems answer this without effort.

1.2 AI System Design Is ML System Design

Keep one sentence in your head for the whole round: **AI system design is machine learning system design.** The discipline has not changed. You start from requirements and metrics. You design the architecture to meet them. You pick the cheapest thing that clears the bar. You design for how the system fails and how you measure it. Then you scale.

What is new is that one core component is non-deterministic. The model does not return the same thing every time, and it can be confidently wrong. That single fact ripples through the whole design, and its most important consequence is this: **evaluation moves from a final gate to the center of the design.** In traditional software, tests run before you ship. In AI systems, evaluation is the thing you build the system around. It is how you decide whether a change is safe, how you catch the model getting quietly worse, and how you convince yourself the system is trustworthy at all. The candidates who put evaluation at the center sound like they have shipped. The ones who bolt it on at the end sound like they have not.

1.3 The 5-Layer Spine

You do not want 10 frameworks for 10 systems. You want one structure strong enough to hang any AI system on, so that when the interviewer changes the domain, you change the contents and keep the shape. That structure is the 5-layer spine, and it is the backbone of every case in this book. It is a place to hang your reasoning, not a template to fill in mechanically. A good answer bends it to the problem.

Layer 5 Optimization routing, caching, and where multi-agent belongs

Layer 4 Production and ops serving, latency, cost, observability, rollback

Layer 3 Evals and guardrails the release gate, moved to the center

Layer 2 The wrapping layer knowledge, tools, memory

Layer 1 The model the one non-deterministic component

Layer 1, the model. The single non-deterministic component. Pick a model that clears the bar for the task, decide how you handle its non-determinism (structured output, a confidence signal, a deterministic step downstream), and know when a cheaper model will do. Most of the interview is not here. The model is the engine; the system is everything around it.

Layer 2, the wrapping layer. Where most of the design lives, and where you should spend most of your time. It is the architecture wrapped around the model, and it is 3 things: **knowledge** (what the system knows, usually retrieval over your own data), **tools** (what the system can do, the typed actions it can take), and **memory** (what it remembers across a turn, a task, and a user). Change the balance of these 3 and you change the system.

Layer 3, evals and guardrails. The consequence of Layer 1. Because the model is non-deterministic, evaluation is the center of the design, not a final gate. This layer is **evals** (how you measure quality offline and online, and catch silent degradation) and **guardrails** (the runtime checks on input and output that keep a bad generation from reaching a user or an action). Treat it as first-class in every case.

Layer 4, production and ops. What it takes to run the thing: serving and scale, the cost, latency, and quality triangle, observability so every step is a traceable span, and a rollback plan. Some systems live or die here; others treat it as table stakes. Knowing which is what experience buys you.

Layer 5, optimization. The last layer, on purpose. Once the system works and you can measure it, you make it faster and cheaper: model routing, caching, and, where the task genuinely justifies it, multi-agent orchestration. Multi-agent lives here because it is an optimization, not a starting point. Reaching for it too early is one of the most common ways a design gets worse.

System design is the work of **composing these 5 into one coherent system** for a specific problem, deciding which layer carries the weight, and defending the choices where they trade off. That is why the same spine produces 10 different systems: in one case Layer 2's tools carry the weight, in another Layer 3's guardrails, in another Layer 4's latency. You will see the same 5 headings in every case and a different subset doing the heavy lifting.

1.4 Scoping with Problem-First

The most common way to fail this round is to start designing too soon. The few minutes before the architecture are where experience shows, and where the design is quietly won or lost.

What is Problem-First? A framework we developed and teach at [LevelUp Labs](#): work backwards from the business problem and the user's pain, scope hard, and only then reach for AI, choosing the smallest intervention that moves a measurable outcome. It is the spine of our [enterprise AI course](#).

Before you architect, pin down 4 things, in order, each in a sentence or two.

- **The user and their pain.** A named role, an actual task, a measurable pain. “Support agents handle 60 tickets a shift, most of them the same routine questions, under a deadline.” Not “users want faster support.”
- **The outcome, written before the system.** What changes when this works, and how would you know. “Deflect the repetitive majority to an agent that resolves them end to end, measured by containment paired with a hard ceiling on wrong actions.”
- **The AI intervention, narrowed until it hurts.** The narrowest useful definition: the pattern (retrieve and answer, extract and decide, draft and review) and the specific capability. This layer is bloated in every failed project. 1 workflow, not 4. Propose and let a human dispose, not full autonomy.
- **The system and safety.** The evaluation set that gates every release, the guardrails, the human approval on high-impact steps, the audit trail, the rollback plan. Named at scoping time so they are not an afterthought.

Ask the clarifying questions that change the design. The good ones are scored. Before you architect, ask the handful whose answers reshape the system: what is the cost of a wrong answer versus a missed one; what is the volume and how repetitive is it; what must a human sign off on; how fresh must the knowledge be and who may see what; what does correct mean here, and can it be checked.

State your assumptions about the data. Every method in the wrapping

layer is chosen for a specific shape of data. Clean structured inputs thin out the extraction layer; messy scans earn a layout-aware parser; a heterogeneous permissioned corpus makes retrieval the hard part. Saying “here is what I am assuming, and here is why it changes the design” tells the interviewer you understand that architecture follows data. Every case in this book has an explicit assumptions step.

Avoid the traps. Solutioning in the problem statement (“build an agent that emails everyone”). Over-scoping three problems into one system that evaluates none of them well. A problem with no owner, so the outcome cannot be defined. No measurable outcome, so Layer 3 collapses.

1.5 How the Book Is Organized

The book is 4 parts. Part 1, this chapter, is the method: what the round tests, the spine, and how to scope. Part 2 is the 10 cases, each worked end to end. Part 3 pulls back to the patterns the cases share and adds an interview-day playbook. Part 4 takes the same 10 cases through the product and strategy lens.

Every case chapter has the same shape, so by the 10th it is muscle memory: the question the way an interviewer asks it, the scoping, the 5 layers walked for that problem, the follow-ups an interviewer pushes on, a real deployment with real numbers, and the code.

This is not only an engineer’s round, so the book is not written for one seat. The design underneath is the same for everyone. What changes is where the weight sits.

- **AI engineers, forward-deployed engineers, and builders.** This is your signature technical round, and the whole conversation is fair game, down to the retrieval strategy and the eval harness. Read Part

1, work the cases in Part 2, run the code, and keep Part 3 as your playbook. You defend the architecture.

- **Product managers, deployment strategists, and consultants.** You get a lighter version of the same conversation inside product-sense and delivery rounds: does AI belong here, what is the model's role, how is it measured, where does a human stay in the loop. Read Part 1, skim the case shapes for the architecture, and go deep in Part 4, which reframes all 10 for your seat. You defend the decision to build it and the metric that proves it worked.
- **Strategy and leadership.** Read across all of it. The value for you is in the scoping, the tradeoffs, and knowing where a design breaks under cost, latency, and compliance, so you can tell an ambitious plan from a buildable one. You defend the sequence and the governance.

1.6 Working the Cases, with the Code

Each case is written to be worked, not read. Close the chapter, answer the question out loud, then open it and compare. Where your answer skipped a layer, that is the layer to study. Where a follow-up caught you flat, that is the rep to run again.

Then run the code. Every case ships a small, provider-agnostic, runnable version of the system, and each chapter points you to the exact folder. Reading the design tells you the shape. Running it tells you where the real decisions live, and an interviewer can tell the difference between a candidate who has read about agents and one who has built one.

The fastest way to learn it is to co-learn with your agent. Point Claude Code, Codex, or any agent harness at the case folder, have it walk you through the system a layer at a time, then change something and watch the design move. A prompt to start with:

Clone `github.com/aishwaryanr/awesome-generative-ai-guide` and open `interview_prep/system-design/customer-support-agent/code`.

Read the code, then walk me through how retrieval, the agent loop, and the guardrails fit together, one layer at a time. Push back where I would make a different choice. Then help me add a live order-status tool, so I can see how the design changes when a new action comes in.

If you would rather read and run everything in one place, the same 10 cases live as a GitHub repo you can clone.

With the round understood, the spine in hand, and the problem scoped, you are ready to design. Part 2 starts with the case that shows the whole shape most cleanly: the customer-support agent.

Get the Full Book

You have just read Part 1: the method for approaching any AI system design interview, the 5-layer spine, and Problem-First scoping.

Part 2 puts it to work on 10 real-world systems, each designed end to end the way the interview actually goes, with runnable companion code for every case.

[Download the full book here](#)